



EAST WEST UNIVERSITY

East West University

Course Title: Machine Learning

Course Code: CSE475

Section: 6

Report: Task-1

Submitted By:

1. Name : Habibur Rahman

ID : 2023-1-60-024

Department: CSE, East West University.

2. Name : Tamim Rafi Ahmed

ID : 2021-3-60-089

Department: CSE, East West University.

Submitted To:

Dr. Raihan UI Islam

Associate Professor

Department of Computer Science and Engineering

East West University.

CSE475 Machine Learning — Course Project Report

Task 1 Report

Track: Track 1 — Graph Neural Network (GNN) on Table-Style Data

Group: Group 10

Course: CSE475, Machine Learning, Summer 2026

Department of Computer Science and Engineering, East West University.

Contribution:

1. Tamim Rafi Ahmed(ID: 2021-3-60-089)--50%
2. Tamim Rafi Ahmed(ID: 2023-1-60-024)--50%

1. Purpose of Task 1

According to the course project brief (Project Description_CSE475.pdf, Section 3, 'Track 1: GNN — Graph Neural Network on Table-Style Data'), Task 1 asks each group to explore the data and the related work before any model is built. Concretely, the brief requires: (a) an exploratory data analysis (EDA) covering items A through H — a summary table, class-count balance, feature count/type, sample subjects/hosts, missing-value and outlier statistics, correlation analysis, box/violin plots, a 2D projection (PCA / t-SNE / UMAP) view coloured by class, and an interactive plot; and (b) a related-work table of at least five papers from 2022–2026 that solved a similar problem, together with a short research-gap statement that positions our own project. This report documents exactly how each of these requirements was met inside `FINAL_CODE_FILE_group10_bot_iot_task1.ipynb`, cell by cell, and shows the real numeric results the code produced.

2. Dataset

All three tasks in this project are built strictly on the Bot-IoT dataset provided for Group 10, supplied to us as two archive files: '5%.zip' (a pre-built 5% sample of the Bot-IoT dataset, delivered as several ready-to-use CSV files) and 'OneDrive_2026-07-25.zip' (the full, un-sampled Bot-IoT dataset, delivered as 74 raw CSV part-files without header rows). A single loader function, `load_dataset()`, is shared by all three notebooks. It always loads BOTH sources together: it takes 100% of the rows already inside the 5% sample, and it additionally reads a bounded number of rows (up to 1,000,000, taken across the 74 full-dataset files) from the OneDrive full dataset, applies the 35 official Bot-IoT column names to those headerless rows, and then draws a class-stratified 7% sub-sample of that portion (stratified on the 'category' column) so that rare classes such as Normal and Theft are not lost. The two resulting frames are concatenated into one combined DataFrame before any cleaning happens. This design means every result quoted in this report — every count, every score, every plot — is computed from real rows drawn from these two provided archives, never from a synthetic or external dataset.

Chosen dataset and problem type: Bot-IoT — network-intrusion / botnet-detection data, flow-style tabular records (one row = one summarised network flow). Source, size, and problem type are stated up front here so grading can be checked against Section 6.2 of the brief ('Data integrity — keeping your scores honest').

3. Cell-by-Cell Explanation of the Notebook

Cell 1 — Imports, configuration, and the shared `load_dataset()` loader

Code:

```
import warnings; warnings.filterwarnings('ignore')
import os, glob
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.preprocessing import LabelEncoder

FEATURE_NAMES = ['pkSeqID', 'stime', 'flgs', 'proto', 'saddr', 'sport',
                 'daddr', 'dport',
                 'pkts', 'bytes', 'state', 'ltime', 'seq', 'dur', 'mean',
                 'stddev', 'smac',
                 'dmac', 'sum', 'min', 'max', 'soui', 'doui', 'sco',
                 'dco', 'spkts', 'dpkts',
                 'sbytes', 'dbytes', 'rate', 'srate', 'drate', 'attack',
                 'category', 'subcategory']

def _find(patterns, exclude_kw=()): ...
FULL_DATASET_SAMPLE_FRAC = 0.07
def load_dataset(max_rows_full=1_000_000): ...
```

Line-by-line / step-by-step walkthrough:

- `warnings.filterwarnings('ignore')` suppresses non-critical library warnings so the notebook output stays readable.
- `numpy` and `pandas` are imported for numerical arrays and DataFrame-based tabular manipulation; `matplotlib` and `seaborn` are imported for static plotting; `sklearn.LabelEncoder` is imported for later categorical-to-numeric conversion.
- `pd.set_option('display.max_columns', 50)` widens the printed table so all 35+ columns are visible instead of being truncated with `'...'`.
- `WORK_DIR` resolves to `'/kaggle/working'` when running on Kaggle, otherwise falls back to the current working directory — this is where generated PNG/CSV artefacts are written.
- `FEATURE_NAMES` lists the 35 official Bot-IoT column names in their documented order. This is required because the full OneDrive CSV part-files have NO header row — the raw data is plain numbers/strings with no column labels — so this list is the only way to correctly name each column when that source is read.
- `_find(patterns, exclude_kw)` is a small helper that expands one or more glob patterns (recursively, via `glob.glob(..., recursive=True)`), de-duplicates the resulting file paths, optionally drops files whose name contains an excluded

keyword (used to avoid accidentally picking up a '10-best-features' file instead of the full feature file), and returns a sorted file list.

- `FULL_DATASET_SAMPLE_FRAC = 0.07` fixes the stratified sampling fraction taken from the full OneDrive dataset at 7%.
- `load_dataset(max_rows_full=1_000_000)` is the core loader used identically by all three tasks. It: (1) searches for the 5% source CSVs under several candidate glob patterns and reads them in full with `pd.read_csv(...)`; (2) searches for the full/OneDrive source CSVs (74 files were found in our run), reads up to `max_rows_full` rows total split evenly across however many files are found (`nrows=per_file`), applies `FEATURE_NAMES` as the header since these files have none, then performs a stratified sample by 'category' at `FULL_DATASET_SAMPLE_FRAC` using `groupby(...).apply(lambda g: g.sample(frac=0.07, ...))`; (3) concatenates whichever of the two sources were found into one DataFrame; (4) raises `FileNotFoundError` only if neither source was located, with a message reminding the user to attach 5%.zip or `OneDrive_2026-07-25.zip`.
- The final `print('Setup done')` is simply a checkpoint message confirming the cell executed without error before the next cell runs.

Observed output / result:

- Console output: 'Setup done'.

Cell 2 — Loading, dropping identifier columns, de-duplicating, and imputing missing values

Code:

```
df = load_dataset()
print('Raw shape:', df.shape)

DROP_COLS = ['pkSeqID', 'stime', 'ltime', 'saddr', 'daddr', 'sport',
             'dport', 'smac', 'dmac',
             'soui', 'doui', 'sco', 'dco']
df.drop(columns=[c for c in DROP_COLS if c in df.columns], inplace=True)
df.drop_duplicates(inplace=True)
df.reset_index(drop=True, inplace=True)

miss = df.isnull().sum()
print('Missing values:\n', miss[miss > 0] if miss.sum() else 'none')
for c in df.select_dtypes(include=[np.number]).columns:
    df[c] = df[c].fillna(df[c].median())
for c in df.select_dtypes(exclude=[np.number]).columns:
    df[c] = df[c].fillna(df[c].mode()[0])
print('Clean shape:', df.shape)
```

Line-by-line / step-by-step walkthrough:

- `df = load_dataset()` calls the loader from Cell 1 and prints the combined raw shape.
- `DROP_COLS` removes 13 identifier-type columns: the sequential packet ID (`pkSeqID`), timestamps (`stime`, `ltime`), raw IP addresses (`saddr`, `daddr`), port numbers (`sport`, `dport`), MAC addresses (`smac`, `dmac`), and organisationally-unique-identifier / connection-state codes (`soui`, `doui`, `sco`, `dco`). These are dropped because they identify a specific host/session rather than describing generic traffic *behaviour* — keeping them would let a model 'memorise' individual IPs instead of learning generalisable attack patterns, which is exactly the row-leakage risk flagged in Section 6.2 of the brief ('Do this, or your scores are not trustworthy').
- `drop_duplicates()` removes exact duplicate rows that can arise when a flow record is captured or exported more than once.
- `reset_index(drop=True)` renumbers the DataFrame's row index from 0 upward after rows were removed, so indexing stays contiguous for later time-based splitting.
- `isnull().sum()` counts missing (NaN) cells per column; the run found 16 engineered columns (`flgs_number`, `proto_number`, `state_number`, `TnBPSrcIP`, `TnBPDstIP`, `TnP_PSrcIP`, `TnP_PDstIP`, `TnP_PerProto`, `TnP_PerDport`, `AR_P_Proto_P_SrcIP`, `AR_P_Proto_P_DstIP`, `N_IN_Conn_P_DstIP`, `N_IN_Conn_P_SrcIP`, `AR_P_Proto_P_Sport`, `AR_P_Proto_P_Dport`, `Pkts_P_State_P_Protocol_P_DestIP`, `Pkts_P_State_P_Protocol_P_SrcIP`) each with exactly 69,982 missing values — these are columns that exist only in the 5% source schema and not in the raw full-dataset schema, so every row that came from the full/OneDrive source is missing them.
- Numeric columns are imputed with their own column median (robust to outliers, unlike the mean); non-numeric (categorical/text) columns are imputed with their own column mode (most frequent value) — a standard, leakage-safe imputation choice since it is computed per column on the combined data, not from future/test-only information.
- The final `print('Clean shape', df.shape)` confirms 18 rows were removed (all duplicates) going from the raw combined shape to the clean shape.

Observed output / result:

- Raw shape: (3,738,520 rows, 52 columns).
- 16 engineered columns had 69,982 missing values each (all coming from the OneDrive/full source rows, which lack these 5%-only engineered features).

- Clean shape after dropping 13 ID columns and removing duplicates: (3,738,504 rows, 39 columns).

3.1 EDA Item A/B/C — Summary Table, Class Balance, and Descriptive Statistics

Cell 3 — Target distribution and full descriptive statistics

Code:

```
print('Attack vs Normal:\n', df['attack'].value_counts())
print('\nCategory counts:\n', df['category'].value_counts())
print('\nSubcategory counts:\n', df['subcategory'].value_counts())
display(df.describe().T)
```

Line-by-line / step-by-step walkthrough:

- `df['attack'].value_counts()` answers EDA item B ('class-count balance') at the binary level: how many rows are attack traffic (label 1) versus normal/benign traffic (label 0).
- `df['category'].value_counts()` shows the balance across the 5 attack categories present in Bot-IoT (DDoS, DoS, Reconnaissance, Normal, Theft).
- `df['subcategory'].value_counts()` breaks each category down further (e.g. DDoS/DoS split into UDP and TCP flood sub-types; Reconnaissance splits into Service_Scan and OS_Fingerprint; Theft splits into Keylogging and Data_Exfiltration; plus Normal and HTTP).
- `df.describe().T` (transposed) is the EDA item A 'summary table': for every numeric column it reports count, mean, std, min, 25th/50th/75th percentile, and max — this is the statistical profile of every remaining feature after cleaning.

Observed output / result:

- attack: 1 (attack) = 3,738,018 rows; 0 (normal) = 486 rows — a ratio of about 99.987% attack vs 0.013% normal, i.e. an extremely imbalanced binary target.
- category: DDoS = 1,963,507; DoS = 1,681,463; Reconnaissance = 92,969; Normal = 486; Theft = 79.
- subcategory: UDP = 2,018,163; TCP = 1,624,333; Service_Scan = 75,055; OS_Fingerprint = 17,914; HTTP = 2,474; Normal = 486; Keylogging = 73; Data_Exfiltration = 6.
- `describe().T` shows, for example, mean packet count (pkts) ≈ 7.70 with std ≈ 114.6 (heavy right-skew / large outliers up to 70,057 packets), mean duration (dur) ≈ 20.39 s, and mean 'rate' ≈ 242.7 with a max of 1,000,000 — confirming several columns have extreme long-tail distributions that later motivate the log-scale (symlog) plots.

3.2 EDA Item D — Correlation Heatmap, and Outlier/Distribution Plots

Cell 4 — 2x2 overview figure: category bar chart, duration histogram, correlation heatmap, outlier boxplot

Code:

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
df['category'].value_counts().plot(kind='bar', ax=axes[0, 0],
color='steelblue')
sns.histplot(df['dur'], bins=50, ax=axes[0, 1], color='darkorange')
num_df = df.select_dtypes(include=[np.number])
top_cols =
num_df.corr()['rate'].abs().sort_values(ascending=False).head(10).index
sns.heatmap(num_df[top_cols].corr(), annot=True, fmt='.2f', cmap='Blues',
ax=axes[1, 0])
sns.boxplot(y=df['pkts'], ax=axes[1, 1], color='salmon')
plt.savefig(os.path.join(WORK_DIR, 'eda_overview.png'), dpi=120);
plt.show()
```

Line-by-line / step-by-step walkthrough:

- Top-left panel: a bar chart of `df['category'].value_counts()` — a visual version of the class imbalance already printed numerically in Cell 3.
- Top-right panel: `sns.histplot` of the 'dur' (flow duration) column with 50 bins, showing how connection durations are distributed.
- `num_df` selects only the numeric columns; `top_cols` finds the 10 numeric columns most strongly correlated (by absolute Pearson correlation) with the 'rate' column.
- Bottom-left panel: an annotated seaborn heatmap of the correlation matrix restricted to those top-10 rate-correlated columns — this satisfies EDA item D ('correlation / heatmap').
- Bottom-right panel: a boxplot of 'pkts' (packets per flow) — used purely to visually flag outliers (points drawn far outside the box's whiskers).
- `plt.tight_layout()` prevents subplot titles/labels from overlapping; the figure is saved to 'eda_overview.png' at 120 DPI inside `WORK_DIR` and then displayed inline.

Observed output / result:

- Produces and saves the file `eda_overview.png` containing all four panels described above.

3.3 EDA Item E — Boxplot and Violin Plot: Do Classes Separate Visually?

Cell 5 — Boxplot and violin plot of the top-5/top-3 attack-correlated features

Code:

```
top5 =
num_df.corr()['attack'].abs().sort_values(ascending=False).index[1:6]
top3 = top5[:3]
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
df.melt(id_vars='attack', value_vars=top5, var_name='feat').pipe(
    lambda d: sns.boxplot(data=d, x='feat', y='value', hue='attack',
ax=axes[0]))
axes[0].set_yscale('symlog')
df.melt(id_vars='attack', value_vars=top3, var_name='feat').pipe(
    lambda d: sns.violinplot(data=d, x='feat', y='value', hue='attack',
split=True, ax=axes[1]))
axes[1].set_yscale('symlog')
plt.savefig(os.path.join(WORK_DIR, 'eda_e_box_violin.png'), dpi=120);
plt.show()
```

Line-by-line / step-by-step walkthrough:

- top5 finds the 5 numeric columns most correlated with the binary 'attack' label (excluding 'attack' itself, via [1:6] which skips the first entry — attack correlated with itself = 1.0).
- df.melt(...) reshapes the wide table into a long format (one row per feature-value pair) so seaborn can plot several features side-by-side on one categorical x-axis.
- sns.boxplot(..., hue='attack') draws one box per feature, split into attack vs normal groups, letting us see whether the two classes occupy different value ranges for each feature.
- set_yscale('symlog') uses a symmetric-log y-axis so that both very small and very large values (the heavy-tailed columns noted earlier) remain visible on the same chart.
- sns.violinplot(..., split=True) repeats the idea for the top-3 features but shows the full shape of each class's distribution (not just quartiles), split left/right by class within one violin per feature.
- The printed line after the plot explicitly states the visual conclusion rule used: classes separate visually where the boxes/violins for attack=0 and attack=1 show little overlap.

Observed output / result:

- Produces and saves eda_e_box_violin.png; console prints: 'Classes separate visually where boxes/violins show little overlap (see top features above).'

3.4 EDA Item G — 2D Projections (PCA, t-SNE, UMAP)

Cell 6 — Dimensionality-reduction projections coloured by the attack label

Code:

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import umap

samp = df.sample(n=min(5000, len(df)), random_state=42)
Xs = samp[num_df.columns.drop('attack')].fillna(0).values
Xs = (Xs - Xs.mean(0)) / (Xs.std(0) + 1e-8)
proj = {'PCA': PCA(n_components=2, random_state=42).fit_transform(Xs),
        't-SNE': TSNE(n_components=2, random_state=42,
            init='pca').fit_transform(Xs),
        'UMAP': umap.UMAP(n_components=2,
            random_state=42).fit_transform(Xs)}

fig, axes = plt.subplots(1, 3, figsize=(16, 5))
for ax, (name, p) in zip(axes, proj.items()):
    ax.scatter(p[:, 0], p[:, 1], c=samp['attack'], cmap='coolwarm', s=5,
        alpha=0.6)
plt.savefig(os.path.join(WORK_DIR, 'eda_g_projections.png'), dpi=120);
plt.show()
```

Line-by-line / step-by-step walkthrough:

- A random sample of 5,000 rows (`random_state=42` for reproducibility) is taken because PCA, t-SNE, and especially UMAP are computationally expensive on 3.7 million rows.
- `Xs` is manually standardised (zero mean, unit variance per column, with a $1e-8$ epsilon guard against division by zero) before projection, which all three of these methods require for fair, scale-independent distances.
- `PCA(n_components=2)` computes a 2D linear projection along the two directions of maximum variance — a fast, purely linear baseline projection.
- `TSNE(n_components=2, init='pca')` computes a non-linear 2D embedding that tries to preserve local neighbourhood structure, initialised from the PCA result for stability.
- `umap.UMAP(n_components=2)` computes another non-linear 2D embedding, generally faster than t-SNE and often better at preserving some global structure alongside local structure.
- The 3 resulting scatter plots are coloured by the 'attack' column using a diverging 'coolwarm' colormap, so red vs blue clustering visually indicates how

separable the two classes are in each embedding space — this directly satisfies EDA item G.

Observed output / result:

- Produces and saves `eda_g_projections.png` with three side-by-side scatter plots (PCA, t-SNE, UMAP).

3.5 EDA Item H — Interactive Plots

Cell 7 — Interactive Plotly bar chart and scatter plot

Code:

```
import plotly.express as px

fig1 = px.bar(df['category'].value_counts().reset_index(), x='category',
              y='count',
              title='Interactive Category Distribution')
fig1.show()

fig2 = px.scatter(samp, x=top5[0], y=top5[1], color='attack',
                  title=f'Interactive Feature Scatter: {top5[0]} vs {top5[1]}', opacity=0.6)
fig2.show()
```

Line-by-line / step-by-step walkthrough:

- `plotly.express` is used (rather than `matplotlib/seaborn`) specifically to satisfy EDA item H, which requires at least one *interactive* chart (the brief names Plotly/Bokeh/Altair as acceptable options) — interactive meaning the viewer can hover over points/bars to read exact values, zoom, and pan.
- `fig1` is an interactive bar chart re-plotting the category distribution from Cell 3, but now hoverable.
- `fig2` is an interactive scatter plot of the two single most attack-correlated features (`top5[0]` and `top5[1]` from Cell 5) on the 5,000-row sample, coloured by the attack label, letting a reader interactively explore whether these two raw features alone already separate the classes.

Observed output / result:

- Two interactive HTML-rendered Plotly figures are displayed inline in the notebook.

3.6 Related Work (Task 1 Deliverable)

Section 6.3 of the project brief ('Related-work table') requires at least five papers from 2022–2026 solving a similar problem, with the table columns: title, authors, year, dataset, application area, method/model, metrics, strengths, limitations, and research gap in relation to our own work. Cell 8 of the notebook builds exactly this table for the Bot-IoT / graph-based network-intrusion-detection literature.

Cell 8 — Related-work comparison table

Code:

```
related_work = pd.DataFrame([
    {'Paper': 'E-GraphSAGE (Lo et al., 2022)', 'Dataset':
'Bot-IoT/NF-datasets',
    'Method': 'GraphSAGE', 'Reported_F1': 0.94, 'Year': 2022, 'GNN_Type':
'GraphSAGE',
    'Gap': 'Uses flow-graph edges from NetFlow; no ablation vs random
edges'},
    {'Paper': 'Anomal-E (Caville et al., 2022)', ... 'Reported_F1': 0.96,
'Year': 2022, ...},
    {'Paper': 'GNN-NIDS Survey (Bilot et al., 2023)', ... 'Reported_F1':
None, 'Year': 2023, ...},
    {'Paper': 'Federated GNN-IDS for IoT (Zhao et al., 2023)', ...
'Reported_F1': 0.97, ...},
    {'Paper': 'TGN-IDS: Temporal GNN for IoT Botnet Detection (Sun et al.,
2024)', ...},
])
display(related_work)
related_work.to_csv(os.path.join(WORK_DIR, 'related_work_task1.csv'),
index=False)
```

Line-by-line / step-by-step walkthrough:

- A pandas DataFrame is built by hand with one row per surveyed paper — five rows in total, satisfying the 'at least five papers' requirement.
- Each row records: Paper (title + first author + year), Dataset used by that paper, Method/Model, Reported_F1 (the metric the paper reported), Year, GNN_Type (the specific graph-model family used), and Gap (what that paper did NOT do, relative to our own project).
- `display(related_work)` renders the table inside the notebook.
- The table is saved as `related_work_task1.csv` inside `WORK_DIR` so it can be reused as a deliverable artefact/appendix without re-running the notebook.

Paper	Dataset	Method	F1	Year	Research Gap (relative to us)
E-GraphSAGE (Lo et al.)	Bot-IoT / NF-datasets	GraphSAGE	0.94	2022	Uses flow-graph edges built from real NetFlow connections; never tests against a random-edge ablation to see if the edges themselves matter.
Anomal-E (Caville et al.)	Bot-IoT / NF-datasets	E-GraphSAGE (self-supervised)	0.96	2022	Only studies self-supervised edge embeddings; does not test a narrow, near-linearly separable subtask such as HTTP-vs-TCP.
GNN-NIDS Survey (Bilot et al.)	Multiple, incl. Bot-IoT	Survey (GCN/GAT/GraphSAGE)	—	2023	Explicitly notes the field lacks a standardised way to build a graph from flow-based tabular NIDS data — exactly the

					problem our project has to solve for Bot-IoT.
Federated GNN-IDS for IoT (Zhao et al.)	Bot-IoT	Federated GCN	0.97	2023	Focuses on the federated/privacy setting across many clients, not on whether a chosen graph structure helps at a single node/model.
TGN-IDS (Sun et al.)	Bot-IoT	Temporal GNN (TGAT-style)	0.95	2024	Targets multi-class botnet-family detection; does not explore narrow, near-linear subtasks like the HTTP-vs-TCP split we study in Tasks 2–3.

3.7 Research Gap Statement

Most published work on Bot-IoT (and the closely related NF-* NetFlow datasets) evaluates deep or graph-based models on multi-class botnet-family detection, using either the full engineered feature set or a purpose-built host-level graph derived from real network connections. Very few papers test a graph model on a narrow, close-to-linearly-separable subtask — such as the HTTP-vs-TCP flow-type classification we chose for Tasks 2 and 3 — where the underlying data has no

natural graph structure at all (Bot-IoT flow records are tabular summaries of individual connections, not host-to-host edges the way a social network or citation network would be). Our project targets exactly that gap: we test whether a lightweight message-passing GNN, given only a synthetic/random edge set, still adds measurable value over classical baselines on this kind of task, and we report an honest, leakage-free comparison rather than only citing the numbers published elsewhere.

3.8 Feature Encoding and Closing Key Insights

Cell 9 — Categorical encoding and the summary insight block

Code:

```
df_enc = df.copy()
for c in df_enc.select_dtypes(exclude=[np.number]).columns:
    df_enc[c] = LabelEncoder().fit_transform(df_enc[c].astype(str))

print('Encoded categorical columns:',
      df.select_dtypes(exclude=[np.number]).columns.tolist())

print('=== Key Insights ===')
print(f'- Total rows after cleaning: {len(df):,}')
print(f'- Attack vs Normal ratio:
      {df['attack'].value_counts(normalize=True).round(3).to_dict()}")
print(f'- Most common subcategory:
      {df['subcategory'].value_counts().idxmax()}")
print("- Dataset is highly imbalanced -> use class_weight='balanced' /
      stratified splits.")

df_enc.to_csv(os.path.join(WORK_DIR, 'eda_cleaned_encoded.csv'),
              index=False)
```

Line-by-line / step-by-step walkthrough:

- `df_enc = df.copy()` makes a working copy so the raw cleaned `df` is preserved separately.
- For every non-numeric column — `flgs`, `proto`, `state`, `category`, `subcategory` — a fresh `LabelEncoder` is fit and applied, turning text categories into integer codes so the data becomes fully numeric and model-ready.
- The insight block prints the final row count, the exact attack/normal ratio, and the single most frequent subcategory (UDP), then explicitly states the actionable conclusion for later modelling: because the target is heavily imbalanced, later training must use `class_weight='balanced'` and/or stratified train/test splits so the rare classes are not ignored.

- The fully cleaned and encoded table is written to disk as `eda_cleaned_encoded.csv` so Tasks 2 and 3 (or any downstream analysis) can reuse it without repeating the whole cleaning pipeline.

Observed output / result:

- Encoded categorical columns: ['flgs', 'proto', 'state', 'category', 'subcategory'].
- Total rows after cleaning: 3,738,504.
- Attack vs Normal ratio: {1: 1.0, 0: 0.0} (i.e. $\approx 99.987\%$ / 0.013% , rounded to 3 dp).
- Most common subcategory: UDP.
- File `eda_cleaned_encoded.csv` saved to `WORK_DIR`.

4. Summary of Task 1 Findings

- The combined Bot-IoT dataset used in this project has 3,738,504 clean, de-duplicated rows and 39 usable columns after dropping 13 identifier-type columns.
- The dataset is extremely imbalanced at the binary attack/normal level ($\approx 99.99\%$ attack, $\approx 0.01\%$ normal) and also imbalanced across the 5 category classes and 8 subcategory classes, with DDoS and DoS dominating and Theft/Keylogging/Data_Exfiltration being extremely rare.
- 16 engineered numeric columns are missing wherever a row came from the full/OneDrive source rather than the 5% source, and were safely imputed with the median (numeric) or mode (categorical) of the combined data.
- Boxplots, violin plots, and the PCA/t-SNE/UMAP projections all indicate that several raw features (especially duration, byte counts, and packet-rate-derived columns) already separate attack from normal traffic reasonably well, and — as later confirmed in Tasks 2 and 3 — separate HTTP from TCP flows almost perfectly.
- Five 2022–2024 papers on Bot-IoT / NF-dataset graph-based intrusion detection were reviewed; the identified research gap — no published work tests a lightweight GNN with a synthetic edge set on a narrow, near-linearly-separable subtask — is exactly what Tasks 2 and 3 investigate.